

Метрики для отслеживания состояния шардов

- Нагрузка по типам операций

```
>_ mongosh: localhost:270... >_ mongosh: shard_1 >_ mongosh: shard_2 +
>_MONGOSH
> const s = db.runCommand({ serverStatus: 1 })
> printjson(s.opcounters)
< {
  insert: Long('521'),
  query: Long('303'),
  update: Long('201'),
  delete: Long('47'),
  getmore: Long('2259'),
  command: Long('9697')
}
shard1 [direct: primary] admin>
```

- Задержки операций (latency)

```
>_ mongosh: localhost:270... >_ mongosh: shard_1 >_ mongosh: shard_2 +
>_MONGOSH
> const s = db.runCommand({ serverStatus: 1 })
> printjson(s.opLatencies)
< {
  commands: {
    latency: Long('2300252'),
    ops: Long('10322'),
    queryableEncryptionLatencyMicros: Long('0')
  },
  reads: {
    latency: Long('734106'),
    ops: Long('2393'),
    queryableEncryptionLatencyMicros: Long('0')
  },
  writes: {
    latency: Long('1002652'),
    ops: Long('504'),
    queryableEncryptionLatencyMicros: Long('0')
  },
  transactions: {
    latency: Long('0'),
    ops: Long('0'),
    queryableEncryptionLatencyMicros: Long('0')
  }
}
shard1 [direct: primary] admin>
```

- Очереди ожидания

>_ mongosh: localhost:270...	>_ mongosh: shard_1	>_ mongosh: shard_2	+
------------------------------	---------------------	---------------------	---

```
>_MONGOSH
> const s = db.runCommand({ serverStatus: 1 })
> printjson(s.globalLock?.currentQueue)
< { total: 0, readers: 0, writers: 0 }
shard1 [direct: primary] admin>
```

Устранение дисбаланса

Причина дисбаланса заключается в неверном выборе шард-ключа (category), из-за чего все товары категории «Электроника» оказались на одном шарде, что привело в свою очередь к большему кол-ву операций на одном шарде и это можно видеть по opcounters и задержкам. Решение заключается в том что бы разбить категорию на несколько диапазонов для распределения их по разным шардам. Для этого меняем шард-ключ на составной добавляя хешированное поле product_id

```
sh.reshardCollection(
"somedb.products",
{ category: 1, product_id: "hashed" }
)
```